а/v запис: https://drive.google.com/file/d/1HvXrqGpLDoYqXD1QEeWhrYreAuadqBA-/view?usp=sharing

Мультиплікативне обернення елемента множини.

Розширений алгоритм **Евкліда.**

[Демонстраційний супровід: Лк7.2 Калькулятор мульт операц GF(p)]

У багатьох Протоколах і алгоритмах е-голосування, під час виконання криптографічних перетворень ненульових даних $a \neq 0$, які належать певному алфавіту GF(p), виникає важлива задача мультиплікативного обернення елемента a є M_p, де M_p - це мультиплікативна група потужності (**p** - 1).

Наприклад, схема Ель-Гамаль для захисту конфіденційності бюлетенів Виборців (В), чи повідомлень Виборчої комісії (ВК).

Або, процедура обчислення пари ключів Виборця (В), чи Виборчої комісії (ВК), приватний та публічний, для отримання сервісів інформаційної безпеки на базі асиметричного алгоритма RSA.

Або, алгоритми постановки та верифікації електронного цифрового підпису (ЕЦП) на бюлетені та повідомлення В та ВК, за тими, чи іншими схемами (Ель-Гамаль, наприклад), чи національними стандартами: DSS-DSA, ДСТУ…

У процесах е-голосування, що є за сутністю своєю інформаційними процесами, виникає потік електронного документообігу, що вимагає використання згаданих вище Протоколів та алгоритмів з криптографічних перетворень.

А у них часто виникає важлива і ресурсоємна задача мультиплікативного обернення елемента «**a**» скінченної множини, числового образа із M_p.

Це є задача пошуку числа ***b*** є M_p, $b \neq 0$, такого, що

$$a \cdot b \equiv 1 \bmod p, \qquad (1)$$

$$a \cdot b - j \cdot p = 1\ . \qquad (1.A)$$

Така задача виникає при потребі ділити якісь дані на певний, ненульовий дільник ***a*** ≠ 0 шляхом множення на елемент ***b*** алфавіта

(множини M_p), мультиплікативно обернений до дільника ***a*** за виразом (1) та (1.A).

Наприклад, це виникає у алгоритмі El-Gamal, чи у Протоколі зі сліпим підписом.

Така задача виникає і при створенні пари ключів за алгоритмом RSA (правда, модуль там є композитний, число Ейлера $\varphi(n)$, що ділиться не тільки на «1», для робочого модуля n).

Переборний метод, шляхом варіювання **j** у виразі (1.A) дає результат за осяжний обсяг часу лише на малих потужностях алфавіту, що певною мірою дозволяє використовувати його у навчальних задачах.

Проте, у криптоаналітика, при малих потужностях алфавіту M_p є спроможність швидко виконати обернення $a \rightarrow b$ і досягти порушення безпеки інформаційної системи, прочитати розшифровані електронні документи: бланки бюлетенів від ВК, або заповнені під час голосування бюлетені Виборця.

Саме тому потужності ***p*** реальних алфавітів у криптоалгоритмах сягають надзвичайно великих значень (елементи множин – числа стають дуже великими), що суттєво знижує шанси криптоаналітиків на успішні атаки шляхом швидкого зламу криптограм у е-голосуванні.

Нагадаємо, що ***p*** може містити 600…1200 десяткових розрядів, цифр. Такі гігантські числа не так вже легко арифметично обчислювати над скінченним полем Галуа GF(p).

Вихід із проблеми обернення чисел на великих алфавітах дає «розширений алгоритм Евкліда», який є природнім продовженням звичайного алгоритма Евкліда для пошуку НСД двох цілих.

АЛГОРИТМ ЕВКЛІДА

Звичайний алгоритм Евкліда призначений для розв'язку проблеми НСД. На його базі отримано розширений алгоритм Евкліда, призначений зокрема для обчислення мультиплікативно оберненого елемента множини за (mod ***p***).

Проблема: Пошук НСД (GCD) двох цілих чисел.

НСД - Найбільший Спільний Дільник.
GCD – Greatest Common Divisor.

Виконується Алгоритмом Евкліда (~300 р. до н.е.)

Приклад 1: a = 205, b = 360.

Шлях пошуку 1: Звичайна факторизація чисел, (якщо це можливо зробити).

Розкладемо a та b на прості фактори-дільники:

a = 205 = 1*5*41

b = 360 = $1*2^3*3^2*5$ НСД(205;360) = 5 ?

Пошук НСД(205;360) перебором дільників є неефективним, вимагає великих витрат ресурсів: часу, обсягу пам'яті, обчислювальної продуктивності, енергії та суттєво залежить від розмірів чисел.

Шлях пошуку 2: **Алгоритм Евкліда НСД, a < b.**

Нехай a = 205, b = 360.

Алгоритм Евкліда вводить у обіг модульні операції над цілими числами.

360 ≡ 155 mod 205 => 360 = 1* 205 + 155

205 ≡ 50 mod 155 => 205 = 1*155 + 50

155 ≡ 5 mod 50 => 155 = 3*50 + **5** - **НСД**(205;360)

Остання ненульова остача і є НСД

50 ≡ **0** mod 5 => 50 = 10*5 + **0**. – індикатор закінчення процедури Евкліда з пошуку НСД

Бачимо цілеспрямовану, регулярну процедуру, яка гарантовано дає результат НСД без виснажливої потреби факторизації двох чисел і порівняльного аналізу їх факторів-дільників.

І це – за наявності проблеми факторизації чисел.

Приклад 2: дано a = 195, b = 252, знайти НСД.

Шлях пошуку 1: Звичайна факторизація чисел, (якщо це можливо зробити).

Розкладемо ***a*** та ***b*** на прості фактори-дільники:

a = 195 = 1*3*5*13

b = 252 = $1*2^2*3^2*7$ НСД(195;252) = 3 ?

Пошук НСД(195;252) перебором дільників є неефективним, вимагає великих витрат ресурсів: часу, обсягу пам'яті, обчислювальної продуктивності, енергії та суттєво залежить від розмірів чисел.

Шлях пошуку 2: **Алгоритм Евкліда НСД, a < b.**

Нехай a = 195, b = 252

Алгоритм Евкліда вводить у обіг модульні операції над цілими числами.

252 ≡ 57 mod 195 => 252 = 1* 195 + 57

195 ≡ 24 mod 57 => 195 = 3*57 + 24

57 ≡ 9 mod 24 => 57 = 2*24 + 9

24 ≡ 6 mod 9 => 24 = 2*9 + 6

9 ≡ 3 mod 6 => 9 = 1*6 + **3** - **НСД**(252;195)

Остання ненульова остача і є НСД

6 ≡ 0 mod 3 => 6 = 2*3 + **0.** – індикатор закінчення процедури Евкліда з пошуку НСД

Бачимо цілеспрямовану, регулярну процедуру, яка гарантовано дає результат НСД без виснажливої потреби факторизації двох чисел і порівняльного аналізу їх факторів-дільників.

І це – за наявності проблеми факторизації чисел.

uuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuu

Нагадаємо проблему, що нас цікавить.

Проблема: Шукаємо (K^{-1}) обернене до K за mod p.

Позначимо число K через **m**.

Наприклад, K = **m** = 446, p = 503, (K^{-1}) = 150

uuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuuu

РОЗШИРЕНИЙ АЛГОРИТМ ЕВКЛІДА

Проблема:

Дано: K = **m** = 446, p = 503, M_p={1,2,3,4,5,…, 502 },

Знайти: (K^{-1}) = ?

Це має назву:

Обернення елемента **m** у термінах теорії чисел.

зрівнюється

[**m**^(-1)] ≡ c mod **p** ,

де **c** є M_p .

Цей алгоритм дозволяє визначити найбільший спільний дільник (НСД) цілих чисел **m** та **p**,

(greatest common divisor, gcd)

а також дозволяє знайти вагові коефіцієнти **x** та **y** такі, що зважена сума

$$m \cdot x + p \cdot y = gcd\ (m, p)\ ,$$

це є зважена сума **m** та **p**, де x та y є цілі вагові коефіцієнти.

Шукаємо мультиплікативно обернений до **m** за mod **p**. Оскільки **m** є M_p, або **m** є GF(p), то **m** < **p**.

Оскільки відомо, що **p** просте, а **m** < **p** , то **m** та **p** – взаємно прості числа, тому можемо прийняти «**1**» як значення НСД, за визначенням.

$$m \cdot x + p \cdot y = 1 = \gcd(m, p) .$$

Візьмемо обидві частини відношення за mod **p**.

$$m \cdot x + p \cdot y \equiv 1 \pmod{p}.$$

«Модуль суми дорівнює сумі модулів». Так стверджує одна із теорем теорії чисел.

Вочевидь, $p \cdot y \equiv 0 \pmod{p}$, тому залишається

$$m \cdot x \equiv 1 \pmod{p}.$$

Таким чином, ***x*** є обернений до **m** за mod **p** і ми можемо застосувати Розширений алгоритм Евкліда (р.а.Е.) для його визначення.

* * * * * **Початок р.а.Е.** * * * * *

A) Позначимо номери кроків алгоритма індексом «**i**» та почнемо з кроку **i** = 0. Частка від ділення, отримана на кожному кроці ***i*** буде позначатися **q**i. Назвемо її «кратність входження» поточного дільника, поточного модуля у поточне ділене на кроці ***i***.

B) Виконуючи кожен крок алгоритму Евкліда, також обчислюємо числове значення допоміжної змінної **s**i.

Значення допоміжної змінної **s**i для перших двох кроків задано, приймаємо: s0 = 0 та s1 = 1.

Для решти кроків «**i**», ми рекурсивно обчислюємо

$$si = \{ s(i-2) - s(i-1) * q(i-2) \} \bmod p.$$

Продовжимо розрахунок si <u>на один крок</u> далі за межі останнього кроку Алгоритма Евкліда.

Алгоритм р.а.Е. починається з «ділення» p на m (m<p; і взяття [p mod m]). Далі на кожному наступному кроці (i+1) виконуємо [$m_{(i)}$ mod $r_{(i)}$], де $m_{(i)}$ та $r_{(i)}$ модуль-дільник та остача із попереднього кроку «i», відповідно.

Якщо останній ненульовий залишок «r» зустрічається на кроці **k**, і якщо цей залишок дорівнює **1** (НСД), то m має обернене, що дорівнює допоміжному числу s(k+2).

Якщо останній ненульовий залишок **не** дорівнює **1** (НСД), то m не має оберненого значення. Цей випадок важливий для композитного модуля ***n***, коли ***n*** не є простим.

Або для **φ(n)**, яке за визначенням є композитним, при обчисленні ключів RSA. Нагадаємо

$$\varphi(n) = (p-1)(q-1).$$

Виконувати алгоритм р.а.Е. треба уважно, слідкуючи за індексами *i* та значеннями поточних компонент «i», si та qi.

x x x x р.а.Е. x x x x

Приклад 1: Знайти обернене до m=446 за mod 503.

Рекусія: $si = \{ s(i-2) - s(i-1) * q(i-2) \} \bmod 503.$

Крок i=0: 503 = 1*446 + 57, s0 = 0 приймаємо.

Крок i=1: 446 = 7*57 + 47, s1 = 1 приймаємо.

Далі: $si = \{ s(i-2) - s(i-1) * q(i-2) \} \bmod 503.$

Крок i=2: 57 = 1*47 + 10,

$s2 = \{0 - 1 * 1\}_{\bmod 503} = (-1)_{\bmod 503} = 502.$

Крок i=3: 47 = 4*10 + 7,

$s3 = \{1-502*7\}_{\bmod 503} = \{1-502*7+503*7\}_{\bmod 503} =$

$= \{1+(503-502)*7\}_{\bmod 503} = \{1+7\}_{\bmod 503} = 8.$

Крок i=4: 10 = 1*7 + 3,

$$si = \{ s(i-2) - s(i-1) * q(i-2) \} \bmod 503.$$

$s4 = \{502-8*1\}_{\bmod 503} = 494.$

Крок i=5: 7=2*3+**1**, **k= i= 5** (**k= 5** останній крок з ненульовою остачею)

$s5 = \{8-494*4\}_{\bmod 503} = \{8-494*4+503*4\}_{\bmod 503} =$

$= \{8+(503-494)*4\}_{\bmod 503} = \{8+9*4\}_{\bmod 503} = 44.$

Крок i=6: 3 = 3*1 + **0**, (крок з нульовою остачею, завершення алгоритма Евк.)

$s6=\{494 - 44 * 1\} \bmod 503 = 450.$

Крок i=7 (додатковий): Остаточно маємо

$s7 = \{44 - 450*2 + 503*2\}_{\bmod 503} = 44 + 106 = \mathbf{150}.$

Кінець алгоритму р.а.Е., $(K^{-1}) = (\mathbf{m}^{-1}) = s7 \equiv 150 \bmod 503.$

Перевірка: $(\mathbf{m}^{-1}) * \mathbf{m} \equiv 150 * 446 \equiv \mathbf{1} \bmod 503,$

або $(K^{-1}) * K \equiv 150 * 446 \equiv \mathbf{1} \bmod 503.$

x x x x p.a.E. x x x x

Приклад 2: Підготовка ключів для алгоритма RSA,

Дано: прості (секретні) p=31, q=47, ключ **e**=7.

Знайти другий ключ **d**, як обернене до **e**=7.

Розв'язок: Модуль шифрування/дешифрування RSA

n = p*q = 31*47= 1457 – не секретний (для даної задачі він не грає ніякої ролі);

число Ейлера ϕ(**n**)=(p-1)*(q -1)=30*46=1380 - секретне;

Задача ставиться наступним чином: Знайти d обернене до **e**=7 за [mod ϕ(**n**)], де ϕ(**n**) =1380.

Виконуємо за розширеним алгоритмом Евкліда.

Нагадаємо, допоміжна змінна si, рекурсія:

$$si = \{ s(\mathbf{i-2}) - s(\mathbf{i-1}) * q(\mathbf{i-2}) \} \bmod 1380.$$

Крок i=0: 1380≡1 mod 7 = 197*7 + **1**, s0 = 0, i= **k** = 0.

(k=i=0 - останній крок з ненульовою остачею)

Крок i=1: 7≡**0** mod 1 = 7*1 + **0**, s1 =1.

(крок з нульовою остачею, завершення звичайного алгоритма Евкліда)

Крок i = 2, додатковий, р.а.Е.:

$s2 = \{ 0 - \mathbf{1} * 197 \} \bmod 1380 = \{-197+\mathbf{1380}\} = 1183.$

Кінець алгоритму р.а.Е., $d = (e^{-1}) = s2 \equiv 1183 \bmod 1380$.

Перевірка: $d * \mathbf{e} \equiv 1183 * 7 \equiv \mathbf{1} \bmod 1380$.

x x x x p.a.E. x x x x

Приклад 3: Підготовка ключів для алгоритма RSA,

Дано: прості (секретні) p=31, q=47, ключ **e**=31.

Знайти другий ключ **d**, як обернене до **e**=31.

Розв'язок: Модуль шифрування/дешифрування RSA

n = p*q = 31*47= 1457 – не секретний (для даної задачі він не грає ніякої ролі);

число Ейлера ϕ(n)=(p-1)*(q -1)=30*46=1380 - секретне;

Задача ставиться наступним чином: Знайти d обернене до e=31 за [mod ϕ(n)], де ϕ(n) =1380.

Виконуємо розширеним алгоритмом Евкліда.

Допоміжна змінна si .

Рекурсія: si = { s(i-2) – s(i-1) * q(i-2) } mod 1380.

Крок i=0: 1380≡16 mod 31 = 44*31+16, s0 =0 приймаємо.

Крок i=1: 31≡15 mod 16 = 1*16+15, s1 =1 приймаємо.

Далі:

Крок i=2: 16≡1 mod 15 = 1*15+1, k = i = 2.

(k=i=2 - останній крок з ненульовою остачею)

si = { s(i-2) – s(i-1) * q(i-2) } mod 1380.

s2 = { 0 –**1** * 44 } mod 1380 = {–44+**1380**} = 1336.

Крок i=3: 15≡**0** mod 1 = 15*1 + **0**, (*останній крок, з нульовою остачею*)

s3 = {1–1336*1} mod 1380 = {1 – 1336 + **1380**} = 45.

Крок i=4, додатковий, р.а.Е.:

s4 = {1336–45*1} mod 1380 = {1336-45} = 1291.

Кінець алгоритму р.а.Е., **d** = (e^{-1}) = s4 ≡ **1291** mod 1380.

Перевірка: d * **e** ≡ 1291 * 31 ≡ **1** mod 1380.

× × × × × × × ×

Розширений алгоритм Евкліда дозволяє знайти значення (m^{-1}), обернене до **m** за модулем цілого числа **p**, за скінченну кількість кроків, де (m^{-1}), **m** є GF(**p**).

р.а.Е. слабше залежить від розмірів чисел у алфавіті ніж інші процедури і швидко, за скінченну кількість кроків, приводить до результату, обернення числа **m**.

Зауваження для RSA і подібних випадків!

Число $\phi(n)$ є композитним, тому випадковий ключ ***e*** треба задавати взаємно-простим із $\phi(n)$.

Якщо остання ненульова остача у р.а.Е. за mod $\phi(n)$ не дорівнює «1», то для ***e*** не існує мультиплікативно оберненого **d** .

Це означає, що ***e*** невірно було задано, воно має НСД із $\phi(n)$ більший за «**1**». Треба перевірити умову для ***e*** і обрати його інше, вірне значення, узгоджене із $\phi(n)$.